

Module 1: Getting Started

Goal: Set up the Python environment, write the first program, understand basic syntax, and get comfortable with the development workflow.

1. What is Python?

Python is a high-level, general-purpose programming language widely recognized for:

- **Simplicity** – Clean and intuitive syntax that feels almost like plain English.
- **Readability** – Code is easy to understand and maintain, even for large projects.
- **Versatility** – Applicable across many domains, from web development to artificial intelligence.
- **Large ecosystem** – Thousands of libraries available via pip.
- **Beginner-friendly** – Gentle learning curve, making it an ideal first language.

Key Milestones:

Milestone	Detail
Creator	Guido van Rossum
Initial Release	1991
Philosophy	“Readability matters” – PEP 20 (The Zen of Python)

Primary Application Areas:

Area	Examples
Data Science & Artificial Intelligence	NumPy, pandas, TensorFlow, PyTorch, scikit-learn
Web Development	Django, Flask, FastAPI
Process Automation & Scripting	Task automation, system administration, web scraping
Software Testing	pytest, Selenium, unittest, behave
Other domains	DevOps (Ansible), Game development (Pygame), Desktop GUI (Tkinter, PyQt)

Example:

```
print("Hello, World!")
```

“Python is an experiment in how much freedom programmers need. Too much freedom lets a program be written in incomprehensible ways; too little freedom makes solutions cumbersome. Python strikes a balance.”

– Guido van Rossum

2. Installing Python

2.1. Check if Python is already installed

Open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type:

```
python --version
```

or

```
python3 --version
```

If you see a version number (e.g., Python 3.12.0), you already have Python.

2.2. Download and install

- Go to python.org/downloads
- Download the latest stable version (3.x)
- **Windows:** During installation, **check the box** “Add Python to PATH” – this is crucial.
- **macOS/Linux:** Use the official installer or package manager (e.g., `brew install python` on macOS, `sudo apt install python3` on Linux).

After installation, verify again:

```
python --version
```

3. Setting Up a Code Editor

Python can be run from the terminal, but a good editor improves productivity.

3.1. Recommended options

Editor	Pros	When to use
VS Code	Free, lightweight, excellent Python extension	Most versatile, recommended for beginners
PyCharm	Powerful IDE, built-in tools	Great for larger projects
IDLE	Comes with Python, very simple	Quick tests, no installation needed

3.2. Installing VS Code

1. Download from code.visualstudio.com
2. Install the **Python extension** by Microsoft
3. Open a folder for your Python projects.

3.3. Installing PyCharm

1. Download from jetbrains.com/pycharm.
 2. Run installer → choose Community (free) or Professional.
 3. Launch PyCharm → **New Project** → select Python interpreter.
 4. Click **Create**.
-

4. Running Python Code

4.1. Interactive Mode (REPL)

Type `python` (or `python3`) in the terminal. You'll see `>>>`. You can write Python commands and see immediate results.

```
>>> print("Hi")
Hi
>>> 2 + 3
```

```
5
>>> exit()    # or Ctrl+Z on Windows, Ctrl+D on macOS/Linux
```

Useful for quick experiments.

4.2. Script Mode (File)

Create a file named `hello.py` with the following content:

```
print("Hello, World!")
```

Then run it from the terminal:

```
python hello.py
```

Or use the “Run” button in your editor.

5. Your First Python Program: `print()` and `input()`

5.1. The `print()` function

Displays output to the console.

```
print("Hello")
print(42)
print("The result is", 5 + 3)
```

- Multiple items can be separated by commas.
- By default, `print()` adds a newline at the end.

5.2. The `input()` function

Reads a line of text from the user and returns it as a string.

```
name = input("What is your name? ")
print("Hello", name)
```

- The argument is the prompt shown to the user.
- The returned value is always a string; convert it if needed (e.g., `int(input("..."))`).

5.3. Simple program combining both

```
# Ask for user's name and age
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Hello", name, "! You are", age, "years old.")
```

5.4. String Formatting: `format()` and **f-strings**

String formatting lets you insert values into a string. Python provides two common ways: the `format()` method and **f-strings** (formatted string literals).

A. The `format()` method

The `format()` method uses `{}` as placeholders.

```
name = "Alice"
age = 25
print("Hello, {}. You are {} years old.".format(name, age))
```

- Values are inserted in order.
- You can change the order with numbered placeholders:

```
print("My name is {1}. I am {0} years old.".format(age, name))
# Output: My name is Alice. I am 25 years old.
```

- You can use named placeholders for clarity:

```
print("Hello, {n}. You are {a} years old.".format(n=name, a=age))
```

Why use format()?

- Works in all Python versions (even older ones before 3.6).
- Allows dynamic formatting strings (e.g., templates read from a file).
- Gives fine control over spacing, alignment, and number formatting (e.g., "{:.2f}".format(3.14159) → "3.14").

B. f-strings (Formatted String Literals) – Python 3.6+

f-strings are a newer, more concise way to format strings. Put an `f` before the string and use `{variable}` directly.

```
name = "Alice"
age = 25
print(f"Hello, {name}. You are {age} years old.")
```

- You can put expressions inside `{}`:

```
print(f"5 + 3 = {5 + 3}")
```

- You can call methods:

```
print(f"Uppercase: {name.upper()}")
```

- You can format numbers:

```
pi = 3.14159
print(f"Pi to 2 decimals: {pi:.2f}") # Pi to 2 decimals: 3.14
```

Why use f-strings?

- Cleaner and easier to read.
- Faster than `format()` because it is evaluated at compile time.

C. Comparison Table

Feature	format()	f-string
Introduced	Python 2.6 / 3.0	Python 3.6
Syntax	"Hello {}".format(name)	f"Hello {name}"
Readability	Good	Excellent
Performance	Slower	Faster
Dynamic template	Yes (from variable)	No (must be literal)
Expression inside	Limited	Full Python expressions

D. Which one should you use?

Situation	Recommendation
You are using Python 3.6+	Prefer f-strings – they are simpler and faster.
You need a dynamic template (e.g., from a file or user input)	Use format() because f-strings require a literal string.
You must support old Python (before 3.6)	Use format() or the older % formatting.

E. Examples side-by-side

```
name = "Bob"
```

```
score = 95.678
```

```
# f-string (modern)
```

```
print(f"{name} scored {score:.1f} points.")
```

```
# format() method
```

```
print("{} scored {:.1f} points.".format(name, score))
```

```
# Both output: Bob scored 95.7 points.
```

6. Comments and Code Readability

Comments are ignored by Python but help humans understand the code.

- **Single-line comment:** start with #

```
# This is a comment
```

```
print("Hello") # This is an inline comment
```

- **Multi-line comment:** use triple quotes (often for docstrings)

```
"""
```

```
This is a
```

```
multi-line comment
```

```
"""
```

Best practices:

- Use comments to explain *why*, not *what* (the code should be self-explanatory).
- Keep comments up-to-date.
- Use meaningful variable names.

7. Hands-On Activities

Activity 1: Environment Check

1. Verify Python installation.
2. Run the Python interpreter and type `print("Python is working!")`.
3. Exit the interpreter and run the same command from a script file.

Activity 2: Personal Greeting

Write a program that:

- Asks the user for their first name and last name.
- Prints a greeting like: "Hello, John Doe!"

Solution:

```
first = input("First name: ")
last = input("Last name: ")
print("Hello,", first, last, "!")
```

Activity 3: Simple Calculator (with integers)

Write a program that asks for two numbers and prints their sum, difference, product, and quotient.

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("Sum:", num1 + num2)
print("Difference:", num1 - num2)
print("Product:", num1 * num2)
print("Quotient:", num1 / num2)
```

Activity 4: Comment Your Code

Take the calculator program and add comments explaining each step.

8. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
python not recognized	Python not added to PATH (Windows)	Reinstall Python and check "Add to PATH"
SyntaxError: invalid syntax	Typo, missing quotes, wrong indentation	Check line carefully, ensure quotes match
NameError: name '...' is not defined	Variable used before assignment or misspelled	Verify spelling and order
input() returns a string but you need a number	Forgot to convert with int() or float()	Use int(input(...))

9. Summary

- Python is a versatile, beginner-friendly language.
 - Install Python and a code editor to start coding.
 - Run Python in interactive mode for tests, and script mode for programs.
 - print() displays output, input() reads user input.
 - Comments improve code readability.
-

10. Additional Resources

- Official Python Tutorial [<https://docs.python.org/3/tutorial>]
 - Python.org Beginner's Guide [<https://www.python.org/about/gettingstarted>]
 - VS Code Python Tutorial [<https://code.visualstudio.com/docs/python/python-tutorial>]
-

Module 2: Python Basics

Goal: Understand Python’s fundamental building blocks: variables, data types, basic operations, and string manipulation.

1. Variables and Data Types

1.1. What is a Variable?

A variable is a named container for storing data. In Python, you don’t need to declare the type; it is inferred from the value.

```
name = "Alice"      # string
age = 25            # integer
height = 1.75       # float
is_student = True   # boolean
```

Rules for variable names:

- Can contain letters, digits, and underscores.
- Must start with a letter or underscore (not a digit).
- Case-sensitive (name ≠ Name).
- Cannot use Python keywords (e.g., if, for, while).

1.2. Data Types Overview

Category	Type	Example	Description
Text Type	str	"Hello", 'Python', """Multi-line"""	Unicode string (text)
Numeric Types	int	42, -5, 0, 1_000_000	Integer (arbitrary precision)
	float	3.14, -0.001, 1.2e3	Double-precision floating point
	complex	3+4j, -2j, 1+0j	Complex number (real + imaginary part)
Sequence Types	list	[1, 2, 3], ['a', 'b']	Mutable ordered collection
	tuple	(1, 2, 3), ('x', 'y')	Immutable ordered collection
	range	range(5), range(1,10,2)	Immutable sequence of numbers (used in loops)
Mapping Type	dict	{'name': 'Alice', 'age': 25}	Key-value pairs (associative array)
Set Types	set	{1, 2, 3}, set([1,2,2])	Unordered collection of unique, mutable items
	frozenset	frozenset([1,2,3])	Immutable version of a set
Boolean Type	bool	True, False	Logical values (subclass of int)
Binary Types	bytes	b'hello', bytes([65,66])	Immutable sequence of bytes (0–255)
	bytearray	bytearray([65,66])	Mutable sequence of bytes
	memoryview	memoryview(b'hello')	Memory-view of binary data (no copy)

Category	Type	Example	Description
None Type	NoneType	None	Represents absence of a value (only one instance: None)

Quick Examples

Numeric types

```
x = 3 + 4j          # complex
print(x.real, x.imag) # 3.0 4.0
```

Sequence types

```
r = range(3)        # 0,1,2
print(list(r))      # [0, 1, 2]
```

Set types

```
fs = frozenset([1,2,2])
print(fs)           # frozenset({1,2})
```

Binary types

```
b = bytes([65,66,67]) # b'ABC'
ba = bytearray(b'hello')
ba[0] = 72             # modify to b'Hello'
print(ba)              # bytearray(b'Hello')
```

NoneType

```
value = None
print(type(value)) # <class 'NoneType'>
```

You can check the type of any variable using type():

```
print(type(age)) # <class 'int'>
```

2. Basic Operations

2.1. Arithmetic Operators

Operator	Description	Example
+	Addition	5 + 3 → 8
-	Subtraction	5 - 3 → 2
*	Multiplication	5 * 3 → 15
/	Division (float)	5 / 2 → 2.5
//	Floor division	5 // 2 → 2
%	Modulo (remainder)	5 % 2 → 1
**	Exponentiation	5 ** 2 → 25

Order of operations: Python follows PEMDAS (parentheses, exponents, multiplication/division, addition/subtraction).

2.2. Comparison Operators

Operator	Description	Example
==	Equal to	5 == 5 → True
!=	Not equal to	5 != 3 → True
>	Greater than	5 > 3 → True
<	Less than	5 < 3 → False
>=	Greater than or equal	5 >= 5 → True
<=	Less than or equal	5 <= 3 → False

2.3. Logical Operators

Operator	Description	Example
and	True if both are true	(5 > 3) and (2 < 4) → True
or	True if at least one is true	(5 > 3) or (2 > 4) → True
not	Reverses the boolean	not (5 > 3) → False

3. Type Conversion (Casting)

Sometimes you need to convert a value from one type to another.

```
# Convert string to int
num_str = "123"
num_int = int(num_str)

# Convert int to float
num_float = float(10)    # 10.0

# Convert float to int (truncates)
num_int = int(3.99)      # 3

# Convert number to string
text = str(42)           # "42"

# Convert to boolean
bool(0)    # False
bool(1)    # True
bool("")   # False
bool("Hi") # True
```

Note: `input()` always returns a string. Use casting to treat input as numbers:

```
age = int(input("Enter your age: "))
```

4. Strings in Depth

Strings are sequences of characters. They can be enclosed in single, double, or triple quotes.

4.1. Indexing and Slicing

Each character has an index (starting from 0). Negative indices count from the end.

```

text = "Python"
# Indices: P y t h o n
#           0 1 2 3 4 5
#          -6 -5 -4 -3 -2 -1

# Access single character
print(text[0]) # 'P'
print(text[-1]) # 'n'

# Slicing: [start:stop:step]
print(text[0:3]) # 'Pyt' (stop index excluded)
print(text[2:]) # 'thon' (from index 2 to end)
print(text[:4]) # 'Pyth' (from start to index 3)
print(text[::2]) # 'Pto' (every 2nd character)
print(text[::-1]) # 'nohtyP' (reverse)

```

4.2. Common String Methods

Method	Description	Example
upper()	Returns uppercase version	"hello".upper() → "HELLO"
lower()	Returns lowercase version	"HELLO".lower() → "hello"
strip()	Removes leading/trailing whitespace	" hi ".strip() → "hi"
replace(old, new)	Replaces substring	"hello".replace("l", "x") → "hexxo"
split(sep)	Splits into list	"a,b,c".split(",") → ["a","b","c"]
join(iterable)	Joins elements with string	"-".join(["a","b","c"]) → "a-b-c"
find(sub)	Returns index of first occurrence	"hello".find("l") → 2
count(sub)	Counts occurrences	"hello".count("l") → 2
startswith(prefix)	Checks if starts with	"hello".startswith("he") → True
endswith(suffix)	Checks if ends with	"hello".endswith("lo") → True

Examples:

```

text = " Hello, World! "
print(text.strip()) # "Hello, World!"
print(text.lower()) # " hello, world! "
print(text.replace("World", "Python")) # " Hello, Python! "
print("apple,banana,cherry".split(",")) # ['apple', 'banana', 'cherry']

```

5. Hands-On Activities

Activity 1: Variable Practice

Create variables for your favorite movie (string), its release year (int), rating (float), and whether you've watched it (bool). Print each variable with a label.

Solution:

```

movie = "Inception"
year = 2010

```

```
rating = 8.8
watched = True

print("Movie:", movie)
print("Year:", year)
print("Rating:", rating)
print("Watched:", watched)
```

Activity 2: Simple Calculator with User Input

Write a program that asks the user for two numbers and displays the sum, difference, product, quotient, and floor division.

Solution:

```
num1 = float(input("First number: "))
num2 = float(input("Second number: "))

print(f"Sum: {num1 + num2}")
print(f"Difference: {num1 - num2}")
print(f"Product: {num1 * num2}")
print(f"Quotient: {num1 / num2}")
print(f"Floor division: {num1 // num2}")
```

Activity 3: Comparison and Logic

Write a program that asks for the user's age and prints:

- "Adult" if age ≥ 18
- "Minor" if age < 18
- Also print "Eligible to vote" if age ≥ 18 and the user is a citizen (ask a boolean question).

Solution:

```
age = int(input("Enter your age: "))
citizen = input("Are you a citizen? (yes/no): ").lower() == "yes"

if age >= 18:
    print("Adult")
    if citizen:
        print("Eligible to vote")
    else:
        print("Not eligible to vote")
else:
    print("Minor")
```

Activity 4: String Manipulation

Given the string: " Python programming is fun! "

4. Remove extra spaces.
5. Convert to uppercase.
6. Replace "fun" with "awesome".
7. Split into words and print the first three words.

Solution:

```
s = "    Python programming is fun!    "
s = s.strip()
s = s.upper()
s = s.replace("FUN", "AWESOME")
words = s.split()
print(words[:3])
```

6. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
TypeError: unsupported operand type(s)	Mixing incompatible types (e.g., "5" + 3)	Cast to appropriate type, e.g., int("5") + 3
ValueError: invalid literal for int()	int() called on a non-numeric string	Validate input or use try/except (later)
String index out of range	Accessing index beyond string length	Check length with len() before indexing
Unexpected results with / vs //	Confusing division types	Remember / always returns float, // floors
input() returns string for numbers	Forgetting to cast	Use int(input()) or float(input())

7. Summary

- Variables store data; Python infers types dynamically.
 - Basic types: int, float, str, bool.
 - Use arithmetic, comparison, and logical operators to manipulate data.
 - Cast between types when necessary, especially when reading user input.
 - Strings are sequences – use indexing, slicing, and methods for powerful text processing.
 - f-strings provide a clean way to embed variables in strings.
-

8. Additional Resources

- [Python Data Types \(Official\)](https://docs.python.org/3/library/stdtypes.html) [https://docs.python.org/3/library/stdtypes.html]
 - [String Methods Documentation](https://docs.python.org/3/library/stdtypes.html#string-methods) [https://docs.python.org/3/library/stdtypes.html#string-methods]
 - [W3Schools Python Strings](https://www.w3schools.com/python/python_strings.asp) [https://www.w3schools.com/python/python_strings.asp]
 - [Real Python: Basic Data Types](https://realpython.com/python-data-types/) [https://realpython.com/python-data-types/]
-

Module 3: Control Flow

Goal: Enable students to control the execution flow of their programs using conditional statements and loops.

1. Conditional Statements: `if`, `elif`, `else`

Conditionals allow the program to execute different blocks of code based on certain conditions.

1.1. The `if` Statement

```
age = 18
if age >= 18:
    print("You are an adult.")
```

- The condition must evaluate to a boolean (True or False).
- The code block is indented (usually 4 spaces).

1.2. `ifelse`

```
age = 16
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

1.3. `ifelifelse`

Use `elif` (short for “else if”) to check multiple conditions.

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}")
```

1.4. Nested Conditionals

You can place conditionals inside other conditionals.

```
if age >= 18:
    if citizen:
        print("Eligible to vote")
    else:
        print("Not eligible to vote")
else:
    print("Too young to vote")
```

Best Practice: Avoid deep nesting by using logical operators or restructuring code.

1.5. Conditional Expressions (Ternary Operator)

A compact way to write simple if-else.

```
status = "Adult" if age >= 18 else "Minor"
```

2. Loops

Loops repeat a block of code multiple times.

2.1. for Loop – Iterating Over Sequences

The for loop is used to iterate over any iterable (list, string, range, etc.).

Over a list

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Over a string

```
for char in "Python":
    print(char)
```

Using range()

```
for i in range(5):           # 0,1,2,3,4
    print(i)
```

```
for i in range(2, 10, 2):    # start, stop, step → 2,4,6,8
    print(i)
```

2.2. while Loop – Condition-Based Repetition

The while loop continues as long as a condition is True.

```
count = 0
while count < 5:
    print(count)
    count += 1    # Important: update the condition to avoid infinite loop
```

2.3. Infinite Loops and How to Avoid Them

An infinite loop runs forever if the condition never becomes False.

This will run forever

```
count = 0
while count < 5:
    print(count)    # forgot to increment count
```

- Always ensure the loop variable is updated.
 - Use Ctrl+C to break out of an infinite loop in the terminal.
-

3. Loop Control Statements

3.1. break – Exit the Loop Entirely

```
for i in range(10):
    if i == 5:
        break
    print(i)    # prints 0,1,2,3,4
```

3.2. continue – Skip the Current Iteration

```
for i in range(5):
    if i == 2:
        continue
    print(i)    # prints 0,1,3,4
```

3.3. else Clause with Loops

The else block executes **only if** the loop completes normally (i.e., not terminated by break).

```
for i in range(5):
    print(i)
else:
    print("Loop finished without break")    # Executes
```

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("This won't run because break occurred")
```

Use case: Useful for searching – if you find an item, break; otherwise, run else.

4. Nested Loops

A loop inside another loop. The inner loop completes all its iterations for each iteration of the outer loop.

<pre>for i in range(3): # outer loop for j in range(2): # inner loop print(f"i={i}, j={j}")</pre>	Output: i=0, j=0 i=0, j=1 i=1, j=0 i=1, j=1 i=2, j=0 i=2, j=1
--	--

5. Simple Pattern Printing

Nested loops are commonly used to print patterns.

5.1. Right-angled Triangle

```
rows = 5
for i in range(1, rows + 1):
    print("*" * i)
```

Output:

```
*  
**
```

5.2. Square

```
size = 5  
for i in range(size):  
    for j in range(size):  
        print("*", end=" ")  
    print()
```

```
***  
****  
*****
```

Output:

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

5.3. Pyramid

```
rows = 5  
for i in range(1, rows + 1):  
    print(" " * (rows - i) + "*" * (2 * i  
- 1))
```

Output:

```
  *  
 ***  
*****  
*****  
*****
```

6. Hands-On Activities

Activity 1: Grading System

Write a program that:

- Asks the user for a score (0–100).
- Prints the letter grade (A: 90+, B: 80-89, C: 70-79, D: 60-69, F: <60).
- If the score is out of range, print an error message.

Solution:

```
score = float(input("Enter your score (0-100): "))  
  
if score < 0 or score > 100:  
    print("Invalid score")  
elif score >= 90:  
    print("A")  
elif score >= 80:  
    print("B")  
elif score >= 70:  
    print("C")  
elif score >= 60:  
    print("D")  
else:  
    print("F")
```

Activity 2: Sum of Numbers

Ask the user for a positive integer n and calculate the sum of all numbers from 1 to n using both a for loop and a while loop.

Solution (for loop):

```
n = int(input("Enter a positive integer:  
"))
```

```
total = 0  
for i in range(1, n + 1):  
    total += i  
print(f"Sum: {total}")
```

Solution (while loop):


```

n = int(input("Enter a positive integer: "))
total = 0
i = 1

while i <= n:
    total += i
    i += 1
print(f"Sum: {total}")

```

Activity 3: Guess the Number Game

- Generate a random number between 1 and 10 (use `random.randint`).
- Let the user guess until they get it right.
- After each guess, tell them if it's too high or too low.
- Count the number of attempts and display it when they guess correctly.
- Use `break` to exit the loop when correct.

Solution:

```

import random

secret = random.randint(1, 10)
attempts = 0

while True:
    guess = int(input("Guess a number (1-10): "))
    attempts += 1
    if guess == secret:
        print(f"Correct! It took you {attempts} attempts.")
        break
    elif guess < secret:
        print("Too low!")
    else:
        print("Too high!")

```

Activity 4: Multiplication Table

Ask the user for a number and print its multiplication table from 1 to 10.

Solution:

```

num = int(input("Enter a number: "))
for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")

```

Activity 5: Pattern Printing

Print the following pattern using nested loops:

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Solution:

```

rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):

```

```
print(j, end=" ")
print()
```

7. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
Infinite loop	Condition never becomes False or loop variable not updated	Ensure the variable changes inside the loop; add a break condition if needed
IndentationError	Mixed tabs/spaces or incorrect indentation	Use consistent spaces (4 per level)
Off-by-one errors	Wrong range endpoints or loop condition	Check start/stop values carefully; test with small numbers
break not working as expected	break placed outside loop or inside nested loop incorrectly	break only exits the innermost loop; use flags if needed
Forgetting to convert input	Using input() directly in numeric comparison	Use int(input()) or float(input())
else with loops confusion	Thinking else runs when condition is False (it runs when no break)	Clarify that else is executed if loop ends normally

8. Summary

- **Conditionals:** if, elif, else allow branching based on boolean expressions.
 - **Loops:**
 - for iterates over sequences or ranges.
 - while repeats while a condition is true.
 - **Control statements:**
 - break exits the loop.
 - continue skips the rest of the current iteration.
 - else executes only if the loop wasn't terminated by break.
 - **Nested loops** enable complex iterations and pattern generation.
-

9. Additional Resources

- [Python Official: if Statement](#)
 - [Python Official: for Statement](#)
 - [Python Official: while Statement](#)
 - [Real Python: Python Loops](#)
-

Module 4: Data Structures

Goal: Understand Python’s built-in data structures and choose the right one for different tasks.

1. Introduction to Data Structures

Data structures are containers that organize and store data efficiently. Python provides four built-in data structures:

Structure	Ordered	Mutable	Allows Duplicates	Use Case
List	Yes	Yes	Yes	Ordered collection, frequent modifications
Tuple	Yes	No	Yes	Fixed data, lightweight, keys for dictionaries
Dictionary	Yes (3.7+)	Yes	Keys: no; Values: yes	Fast lookups by key
Set	No	Yes	No	Unique items, membership tests

2. Lists

Lists are ordered, mutable sequences. They can hold items of different types.

2.1. Creating Lists

```
empty_list = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
nested = [[1, 2], [3, 4]]
```

2.2. Accessing Elements (Indexing & Slicing)

Same as string indexing.

```
fruits = ["apple", "banana", "cherry", "date"]
print(fruits[0])      # "apple"
print(fruits[-1])     # "date"
print(fruits[1:3])    # ["banana", "cherry"]
print(fruits[:2])     # ["apple", "cherry"]
```

2.3. Common List Methods

Method	Description	Example
append(item)	Adds item to the end	fruits.append("elderberry")
insert(index, item)	Inserts at position	fruits.insert(1, "blueberry")
extend(iterable)	Adds all items from iterable	fruits.extend(["fig", "grape"])
remove(item)	Removes first occurrence	fruits.remove("banana")

Method	Description	Example
<code>pop(index)</code>	Removes and returns item at index (default last)	<code>last = fruits.pop()</code>
<code>index(item)</code>	Returns index of first occurrence	<code>pos = fruits.index("cherry")</code>
<code>count(item)</code>	Counts occurrences	<code>cnt = fruits.count("apple")</code>
<code>sort()</code>	Sorts in place	<code>numbers.sort()</code>
<code>reverse()</code>	Reverses in place	<code>numbers.reverse()</code>
<code>clear()</code>	Removes all items	<code>fruits.clear()</code>
<code>copy()</code>	Returns shallow copy	<code>fruits2 = fruits.copy()</code>

Examples:

```
colors = ["red", "blue", "green"]
colors.append("yellow")
colors.insert(1, "orange")
print(colors) # ["red", "orange", "blue", "green", "yellow"]

colors.sort()
print(colors) # ["blue", "green", "orange", "red", "yellow"]
```

2.4. List Comprehensions

A concise way to create lists.

```
# Create list of squares
squares = [x**2 for x in range(1, 6)] # [1, 4, 9, 16, 25]

# With condition
evens = [x for x in range(1, 11) if x % 2 == 0] # [2, 4, 6, 8, 10]
```

2.5. Iterating Over Lists

```
for fruit in fruits:
    print(fruit)

# With index
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

3. Tuples

Tuples are **immutable** sequences. Once created, they cannot be changed.

3.1. Creating Tuples

```
empty = ()
point = (10, 20)
single = (5,) # Note the comma; otherwise it's just int
```

3.2. Accessing Elements (Indexing & Slicing)

Same as lists, but no modification.

```
point = (10, 20)
print(point[0])    # 10
```

3.3. Why Use Tuples?

- **Immutability** ensures data cannot be accidentally changed.
- **Performance** – slightly faster than lists.
- **Dictionary keys** – only immutable types can be keys (tuples can, lists cannot).
- **Return multiple values** from functions.

3.4. Tuple Packing and Unpacking

```
# Packing
coordinates = 10, 20, 30    # parentheses optional
```

```
# Unpacking
x, y, z = coordinates        # x=10, y=20, z=30
```

```
# Swapping variables
a, b = b, a
```

3.5. Methods

Tuples have only two methods: `count()` and `index()`.

4. Dictionaries

Dictionaries store key-value pairs. Keys must be immutable (string, number, tuple). Values can be any type.

4.1. Creating Dictionaries

```
empty = {}
person = {"name": "Alice", "age": 25, "city": "New York"}
```

```
# Using dict() constructor
person = dict(name="Alice", age=25)
```

4.2. Accessing Values

```
print(person["name"])    # "Alice"
print(person.get("age")) # 25
print(person.get("country", "Unknown")) # "Unknown" (default)
```

4.3. Modifying Dictionaries

```
person["age"] = 26        # update
person["job"] = "Engineer" # add
del person["city"]        # delete
```

```
# Remove and return value
age = person.pop("age")
```

```
# Get and remove last inserted item (Python 3.7+)
item = person.popitem()
```

4.4. Dictionary Methods

Method	Description
keys()	Returns view of keys
values()	Returns view of values
items()	Returns view of (key, value) pairs
update(other_dict)	Merges another dictionary
clear()	Removes all items
copy()	Shallow copy

Examples:

```
for key in person.keys():
    print(key)

for value in person.values():
    print(value)

for key, value in person.items():
    print(f"{key}: {value}")
```

4.5. Dictionary Comprehensions

```
squares = {x: x**2 for x in range(1, 6)}    # {1:1, 2:4, 3:9, 4:16, 5:25}
```

5. Sets

Sets are unordered collections of **unique** elements. They support mathematical set operations.

5.1. Creating Sets

```
empty = set()          # Note: {} creates empty dict
numbers = {1, 2, 3, 3}  # {1, 2, 3} (duplicates removed)
from_list = set([1, 2, 2, 3]) # {1, 2, 3}
```

5.2. Adding and Removing Elements

```
numbers.add(4)
numbers.remove(2)    # raises KeyError if not present
numbers.discard(5)   # no error if not present
numbers.pop()        # removes and returns an arbitrary element
numbers.clear()
```

5.3. Set Operations

Operation	Method	Operator
Union	union()	\
Intersection	intersection()	&
Difference	difference()	-
Symmetric Difference	symmetric_difference()	^
Subset	issubset()	<=
Superset	issuperset()	>=

Examples:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
print(A | B)    # {1,2,3,4,5,6}
print(A & B)    # {3,4}
print(A - B)    # {1,2}
print(A ^ B)    # {1,2,5,6}
```

5.4. Set Comprehensions

```
squares = {x**2 for x in range(1, 6)}    # {1,4,9,16,25}
```

6. Choosing the Right Data Structure

Task	Recommended Structure
Store ordered collection, allow duplicates, need modifications	List
Store fixed data, need immutability (e.g., dictionary keys)	Tuple
Look up values by unique key	Dictionary
Ensure uniqueness, test membership efficiently	Set

7. Hands-On Activities

Activity 1: List Manipulation

Create a list of 5 numbers. Perform the following:

- Append a new number at the end.
- Insert a number at index 2.
- Remove the third element.
- Sort the list in descending order.
- Create a new list with squares of all numbers using list comprehension.

Solution:

```
nums = [5, 2, 8, 1, 9]
nums.append(3)
nums.insert(2, 7)
nums.pop(2)    # or del nums[2]
nums.sort(reverse=True)
squares = [x**2 for x in nums]
print(squares)
```

Activity 2: Tuple Unpacking

Write a function that returns both the minimum and maximum of a list. Use tuple unpacking to assign the results.

Solution:

```
def min_max(numbers):
    return min(numbers), max(numbers)

nums = [4, 7, 2, 9, 5]
minimum, maximum = min_max(nums)
print(f"Min: {minimum}, Max: {maximum}")
```

Activity 3: Set Operations

Given two lists, find:

- Common elements (intersection)
- Elements in list1 but not in list2 (difference)

- All unique elements from both lists (union)

```
set1 = set(list1)
set2 = set(list2)
```

```
common = set1 & set2
diff = set1 - set2
union = set1 | set2
```

```
print("Common:", common)
print("Only in list1:", diff)
print("All unique:", union)
```

Solution:

```
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
```

Activity 4: Phonebook using Dictionary

Create a phonebook dictionary (name → phone number). Provide a menu to:

- Add contact
- Search contact
- Delete contact
- Display all contacts

Solution:

```
phonebook = {}

while True:
    print("\n1. Add contact\n2. Search\n3. Delete\n4. Display\n5. Exit")
    choice = input("Choose an option: ")

    if choice == "1":
        name = input("Name: ")
        number = input("Phone: ")
```

```
        phonebook[name] = number
        print("Added.")
    elif choice == "2":
        name = input("Name: ")
        print(phonebook.get(name, "Not found"))
    elif choice == "3":
        name = input("Name: ")
        if name in phonebook:
            del phonebook[name]
            print("Deleted.")
        else:
            print("Not found.")
    elif choice == "4":
        for name, number in phonebook.items():
            print(f"{name}: {number}")
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

Activity 5: Word Frequency Counter

Write a program that reads a sentence from the user and counts the frequency of each word (case-insensitive, ignore punctuation).

Solution:

```
sentence = input("Enter a sentence: ").lower()
# Remove punctuation (simple approach)
for p in ".,!?:;":
```

```
    sentence = sentence.replace(p, "")

words = sentence.split()
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1

for word, count in freq.items():
    print(f"{word}: {count}")
```


8. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
<code>TypeError: unhashable type: 'list'</code>	Trying to use a list as dictionary key or set element	Use tuple instead if immutability is needed
<code>KeyError</code>	Accessing missing dictionary key with <code>[]</code>	Use <code>.get()</code> to provide default or check with <code>in</code>
Modifying list while iterating	Skipping elements or causing runtime error	Iterate over a copy (<code>for item in list[:]:</code>) or use list comprehension
Confusion between <code>list.remove()</code> and <code>list.pop()</code>	<code>remove()</code> deletes by value, <code>pop()</code> by index	Remember: <code>remove()</code> raises error if value missing; <code>pop()</code> returns removed item
Duplicates in set not automatically handled when converting from list	Not realizing set removes duplicates	Use <code>set(list)</code> to deduplicate
Order of items in set or dict not guaranteed (pre-3.7)	Assuming order	In Python 3.7+, dict maintains insertion order; sets are unordered. Use <code>sorted()</code> if order matters.

9. Summary

- **Lists** – ordered, mutable, allow duplicates. Use for general collections.
 - **Tuples** – ordered, immutable. Use for fixed data, dictionary keys, multiple returns.
 - **Dictionaries** – key-value mappings, fast lookups. Keys must be immutable.
 - **Sets** – unordered, unique elements. Use for membership tests and set operations.
 - Choose the right structure based on whether you need order, mutability, uniqueness, and key-value mapping.
-

10. Additional Resources

- Python Official: Data Structures [<https://docs.python.org/3/tutorial/datastructures.html>]
 - Real Python: Lists and Tuples [<https://realpython.com/python-lists-tuples>]
 - Real Python: Dictionaries [<https://realpython.com/python-dicts>]
 - Real Python: Sets [<https://realpython.com/python-sets>]
-

Module 5: Functions

Goal: Learn how to write reusable, modular code using functions, understand scope, and explore functional programming basics.

1. What is a Function?

A function is a reusable block of code that performs a specific task. Functions help:

- **Avoid repetition** (DRY – Don't Repeat Yourself)
- **Organize code** into logical sections
- **Make testing easier** by isolating functionality

1.1. Defining a Function

```
def function_name(parameters):  
    """Optional docstring"""  
    # function body  
    return value # optional
```

Example:

```
def greet():  
    print("Hello, World!")
```

```
greet() # call the function
```

1.2. Parameters and Arguments

- **Parameters** are variables listed in the function definition.
- **Arguments** are values passed when calling the function.

```
def greet(name): # name is a parameter  
    print(f"Hello, {name}!")
```

```
greet("Alice") # "Alice" is an argument
```

2. Types of Arguments

2.1. Positional Arguments

Arguments are passed in the same order as parameters.

```
def add(a, b):  
    return a + b
```

```
result = add(3, 5)    # a=3, b=5
```

2.2. Keyword Arguments

Arguments are passed by parameter name, order doesn't matter.

```
result = add(b=5, a=3)
```

2.3. Default Parameters

Assign a default value to a parameter. If omitted, the default is used.

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")
```

```
greet("Alice")           # Hello, Alice!  
greet("Bob", "Hi")       # Hi, Bob!
```

Important: Default parameters must come after non-default parameters.

2.4. Variable-Length Arguments

- args – accepts any number of positional arguments (as a tuple)
- kwargs – accepts any number of keyword arguments (as a dictionary)

```
def print_args(*args, **kwargs):  
    print("Positional:", args)  
    print("Keyword:", kwargs)
```

```
print_args(1, 2, 3, name="Alice", age=25)  
# Positional: (1, 2, 3)  
# Keyword: {'name': 'Alice', 'age': 25}
```

3. Return Values

A function can return a value using the return statement. If no return, the function returns None.

```
def add(a, b):  
    return a + b
```

```
result = add(4, 6)  
print(result)    # 10
```

```
def no_return():  
    print("This returns None")
```

```
value = no_return()
print(value)    # None
```

3.1. Returning Multiple Values

Return a tuple; unpack at call time.

```
def min_max(numbers):
    return min(numbers), max(numbers)

minimum, maximum = min_max([4, 2, 9, 1])
print(minimum, maximum)    # 1 9
```

4. Scope of Variables

Scope determines where a variable can be accessed.

4.1. Local Scope

Variables defined inside a function are local to that function.

```
def my_func():
    x = 10    # local variable
    print(x)

my_func()
# print(x)    # NameError: x is not defined
```

4.2. Global Scope

Variables defined outside any function are global. Use the `global` keyword to modify them inside a function.

```
count = 0    # global

def increment():
    global count
    count += 1

increment()
print(count)    # 1
```

Best Practice: Avoid modifying globals; use parameters and return values instead.

4.3. `nonlocal` for Nested Functions

`nonlocal` allows modifying variables in the outer (enclosing) scope.

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x += 5
    inner()
    print(x)    # 15
```

5. Lambda Functions

Lambda functions are small anonymous functions defined with the `lambda` keyword.

```
# Syntax: lambda arguments: expression
square = lambda x: x ** 2
print(square(5))    # 25

# Often used with map, filter, sort
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))    # [1,4,9,16]
evens = list(filter(lambda x: x % 2 == 0, numbers))    # [2,4]
```

6. Docstrings

Docstrings (documentation strings) describe what a function does. They are written as the first statement in a function body.

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b

print(help(add))    # Shows the docstring
print(add.__doc__)  # Access directly
```

Convention: Use triple-quoted strings and follow PEP 257.

7. Recursion (Optional)

Recursion is a function calling itself. It must have a base case to stop.

```
def factorial(n):
    """Return n! (n factorial) using recursion."""
    if n <= 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))    # 120
```

When to use: Problems that can be broken into similar sub-problems (tree traversal, divide-and-conquer). **Caution:** Python has recursion limit (~1000).

8. Hands-On Activities

Activity 1: Simple Calculator Functions

Create functions for addition, subtraction, multiplication, and division. Then write a main

program that asks the user for two numbers and an operation, and calls the appropriate function.

Solution:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b == 0:  
        return "Error: Division by zero"  
    return a / b
```

Activity 2: Greeting with Default Parameter

Write a function `greet(name, greeting="Hello")` that prints a greeting. Test it with and without the greeting argument.

Solution:

```
# Main program  
num1 = float(input("First number: "))  
num2 = float(input("Second number: "))  
op = input("Operation (+, -, *, /): ")  
  
if op == '+':  
    print(add(num1, num2))  
elif op == '-':  
    print(subtract(num1, num2))  
elif op == '*':  
    print(multiply(num1, num2))  
elif op == '/':  
    print(divide(num1, num2))  
else:  
    print("Invalid operation")
```

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Alice")  
greet("Bob", "Hi")
```

Activity 3: Variable-Length Arguments Sum

Write a function `sum_all(*args)` that returns the sum of all numbers passed.

Solution:

```
def sum_all(*args):  
    return sum(args)  
  
print(sum_all(1, 2, 3, 4))    # 10  
print(sum_all(5, 10))        # 15
```

Activity 4: Palindrome Check

Write a function `is_palindrome(s)` that returns True if the string is a palindrome (ignoring case and spaces). Use it in a main program.

Solution:

```
def is_palindrome(s):  
    # Remove spaces and convert to lowercase  
    cleaned = ''.join(s.split()).lower()
```

```
    return cleaned == cleaned[::-1]
```

```
text = input("Enter a string: ")  
if is_palindrome(text):  
    print("It's a palindrome!")  
else:  
    print("Not a palindrome.")
```

Activity 5: Recursive Fibonacci

Write a recursive function `fibonacci(n)` that returns the nth Fibonacci number (0, 1, 1, 2, 3, 5, ...). Test with `n=10`.

Solution:

```
def fibonacci(n):  
    if n <= 1:  
        return n
```

```
)    return fibonacci(n-1) + fibonacci(n-2)
                                           print(fibonacci(10))    # 55
```

Note: Discuss performance; iterative version is more efficient.

9. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
NameError: name 'x' is not defined	Trying to use a local variable outside its function	Return the value and assign to a variable in the caller
TypeError: missing 1 required positional argument	Not enough arguments passed	Check function signature; provide default values if appropriate
Modifying global variable unintentionally	Forgetting global inside function	Use global if needed, but better to pass as parameter and return
Mutable default arguments	Using a mutable object (e.g., list) as default; changes persist	Use None as default and create new object inside
RecursionError: maximum recursion depth exceeded	Missing base case or too deep recursion	Add base case; consider iterative alternative

10. Summary

- **Functions** encapsulate reusable code.
 - Use parameters to pass data; return values to send data back.
 - **Argument types:** positional, keyword, default, args, *kwargs.
 - **Scope:** local variables exist only inside function; global variables need global to modify.
 - **Lambda functions** are anonymous, short functions.
 - **Docstrings** document your code.
 - **Recursion** is a function calling itself; must have a base case.
-

11. Additional Resources

- Python Official: Functions [<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>]
 - Real Python: Defining Your Own Python Function [<https://realpython.com/defining-your-own-python-function/>]
 - PEP 257 – Docstring Conventions [<https://www.python.org/dev/peps/pep-0257/>]
 - Python Scope & LEGB Rule [<https://realpython.com/python-scope-legb-rule/>]
-

Module 6: Modules and Packages

Goal: Learn how to structure Python projects, reuse code across files, and leverage the rich ecosystem of third-party libraries.

1. What are Modules and Packages?

- **Module:** A single Python file (.py) that contains definitions (functions, classes, variables) and statements.
- **Package:** A directory containing multiple modules and an `__init__.py` file (can be empty). Packages allow hierarchical organization.

Why use modules/packages?

- Reusability: write once, use in multiple projects.
 - Namespace management: avoid name collisions.
 - Maintainability: split large codebases into manageable pieces.
-

2. Importing Modules

2.1. Basic Import

```
import math
```

```
print(math.sqrt(16))    # 4.0
```

2.2. Import Specific Items

```
from math import sqrt, pi
```

```
print(sqrt(25))         # 5.0  
print(pi)               # 3.1415...
```


2.3. Import with Alias

```
import numpy as np    # common alias
from math import sqrt as square_root

print(square_root(9))    # 3.0
```

2.4. Import All (Not Recommended)

```
from math import *    # pollutes namespace
```

Avoid – makes code harder to read and can cause name conflicts.

2.5. Importing Your Own Modules

If you have a file `mymodule.py` in the same directory:

```
import mymodule

# or
from mymodule import some_function
```

3. The Python Standard Library

Python comes with a rich standard library. Here are some commonly used modules:

Module	Description	Example
math	Mathematical functions	<code>math.sin()</code> , <code>math.pi</code>
random	Generate random numbers	<code>random.randint(1,10)</code>
datetime	Date and time handling	<code>datetime.datetime.now()</code>
os	Operating system interface	<code>os.getcwd()</code> , <code>os.listdir()</code>
sys	System-specific parameters	<code>sys.argv</code> , <code>sys.exit()</code>
json	JSON encoding/decoding	<code>json.loads()</code> , <code>json.dumps()</code>
re	Regular expressions	<code>re.search()</code> , <code>re.findall()</code>
collections	Advanced data structures	<code>defaultdict</code> , <code>Counter</code>
pathlib	Object-oriented filesystem paths	<code>Path('file.txt').read_text()</code>

Example:

```
import random
import datetime

print(random.randint(1, 100))
print(datetime.datetime.now())
```

4. Creating Your Own Modules

4.1. Simple Module

Create a file `mymath.py`:

```
# mymath.py
def add(a, b):
    return a + b
```

```
def multiply(a, b):  
    return a * b
```

```
PI = 3.14159
```

In another file (or interactive session):

```
import mymath
```

```
print(mymath.add(5, 3))      # 8  
print(mymath.multiply(4, 2)) # 8  
print(mymath.PI)            # 3.14159
```

4.2. Module Search Path

Python looks for modules in:

13. The current directory.
14. Directories listed in PYTHONPATH.
15. Installation-dependent default paths (e.g., site-packages).

You can view the search path with:

```
import sys  
print(sys.path)
```

5. The if __name__ == "__main__" Idiom

When a Python file is run directly, its __name__ variable is set to "__main__". When imported, __name__ is set to the module's name. This idiom allows code to be executed only when the file is run directly, not when imported.

```
# mymodule.py  
def greet(name):  
    return f"Hello, {name}!"  
  
if __name__ == "__main__":  
    # This code runs only if executed directly  
    print("Running as main script")  
    print(greet("World"))
```

Use case: Write modules that can be both imported and executed as standalone scripts.

6. Packages

A package is a directory containing an __init__.py file (can be empty) and multiple module files.

6.1. Package Structure

```
my_package/  
    __init__.py  
    module1.py  
    module2.py  
    sub_package/  
        __init__.py  
        module3.py
```

6.2. Creating a Package

16. Create a directory (e.g., shapes/).
17. Add an empty __init__.py.
18. Add module files.

Example:

```
shapes/  
    __init__.py  
    circle.py  
    rectangle.py
```

```
**circle.py:**
```

```
def area(radius):  
    return 3.14159 * radius ** 2
```

```
**rectangle.py:**
```

```
def area(length, width):  
    return length * width
```

6.3. Using the Package

```
from shapes import circle, rectangle
```

```
print(circle.area(5))  
print(rectangle.area(4, 6))
```

Or import inside `__init__.py` to expose functions directly.

7. Installing Third-Party Packages with pip

pip is Python's package installer.

7.1. Basic Commands

Install a package

```
pip install package_name
```

Install a specific version

```
pip install package_name==1.2.3
```

Upgrade a package

```
pip install --upgrade package_name
```

Uninstall

```
pip uninstall package_name
```

List installed packages

```
pip list
```

Show package details

```
pip show package_name
```

7.2. Popular Third-Party Packages

Package	Purpose
requests	HTTP requests
numpy	Numerical computing
pandas	Data analysis
matplotlib	Plotting

Package	Purpose
flask	Web development
django	Full-stack web framework

Example:

```
pip install requests
```

```
import requests
```

```
response = requests.get("<https://api.github.com>")  
print(response.status_code)
```

8. Virtual Environments

Virtual environments isolate project dependencies, avoiding conflicts between projects.

8.1. Creating a Virtual Environment

```
# Using venv (built-in)
```

```
python -m venv myenv
```

```
# Activate on Windows
```

```
myenv\Scripts\activate
```

```
# Activate on macOS/Linux
```

```
source myenv/bin/activate
```

8.2. Managing Dependencies

Once activated, install packages with pip. To export requirements:

```
pip freeze > requirements.txt
```

To install from a requirements file:

```
pip install -r requirements.txt
```

8.3. Deactivating

```
deactivate
```

Best Practice: Always use a virtual environment for projects.

9. Hands-On Activities

Activity 1: Explore the Standard Library

Write a program that:

- Generates a random number between 1 and 100.
- Gets the current date and time.
- Creates a directory named test_dir using os (or pathlib).

Solution:

```
import random  
import datetime  
import os  
  
print(random.randint(1, 100))
```

```
print(datetime.datetime.now())
os.mkdir("test_dir")
print("Directory created")
```

Activity 2: Create and Use Your Own Module

19. Create a file `string_utils.py` with functions:
- `reverse_string(s)` - returns reversed string.
 - `count_vowels(s)` - returns vowel count.
20. In another file, import and use these functions.

Solution:

```
# string_utils.py
def reverse_string(s):
    return s[::-1]
```

```
def count_vowels(s):
    vowels = "aeiouAEIOU"
    return sum(1 for char in s if char in vowels)

# main.py
import string_utils

text = input("Enter a string: ")
print("Reversed:", string_utils.reverse_string(text))
print("Vowel count:", string_utils.count_vowels(text))
```

Activity 3: Use `if __name__ == "__main__"` for Testing

Modify `string_utils.py` to include a test block that runs only when executed directly.

Solution:

```
# string_utils.py
def reverse_string(s):
    return s[::-1]

def count_vowels(s):
    vowels = "aeiouAEIOU"
```

```
    return sum(1 for char in s if char in vowels)

if __name__ == "__main__":
    # Test the functions
    print(reverse_string("hello"))    # olleh
    print(count_vowels("Hello World"))    # 3
```

Activity 4: Create a Simple Package

Create a package `geometry` with modules:

- `circle.py`: function `area(radius)`
- `rectangle.py`: function `area(length, width)`
- `triangle.py`: function `area(base, height)`

Then write a script that imports and uses them.

Solution Structure:

```
geometry/
  __init__.py
  circle.py
```

```

rectangle.py
triangle.py

**circle.py:**

import math
def area(radius):
    return math.pi * radius ** 2

**rectangle.py:**

def area(length, width):
    return length * width

**triangle.py:**

def area(base, height):
    return 0.5 * base * height

**main.py:**

from geometry import circle, rectangle, triangle

print(circle.area(5))
print(rectangle.area(4, 6))
print(triangle.area(3, 4))

```

Activity 5: Install and Use an External Package

Install requests (if not installed) and write a program that fetches a URL and prints the status code and first 200 characters of the response.

Solution:

```

import requests

url = input("Enter URL: ")
response = requests.get(url)
print(f"Status code: {response.status_code}")
print("First 200 chars:")
print(response.text[:200])

```

10. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
ModuleNotFoundError	Module not in search path or not installed	Check <code>pip list</code> , install if needed; verify file location
ImportError: cannot import name	Name misspelled or not defined in module	Check spelling and that the name exists
Package installed but still ModuleNotFoundError	Wrong Python environment	Ensure you're using the correct environment (activate virtual env)
pip not recognized	Python not added to PATH (Windows)	Reinstall Python with "Add to PATH" or use <code>python -m pip</code>

Problem	Likely Cause	Solution
<code>__init__.py</code> missing in package	Python 3.3+ still needs it for namespace packages? Not required, but often used for initialization	Add empty <code>__init__.py</code> to make directory a regular package

11. Summary

- **Modules** are `.py` files; **packages** are directories with `__init__.py`.
 - Use `import`, `from ... import`, and aliases to bring code into your namespace.
 - The standard library provides many useful modules.
 - `if __name__ == "__main__":` allows code to run only when the file is executed directly.
 - `pip` is the package manager for installing third-party libraries.
 - **Virtual environments** isolate project dependencies.
 - Organize code into modules and packages to keep projects clean and reusable.
-

12. Additional Resources

- [Python Official: Modules](#)
 - [Python Standard Library](#)
 - [Real Python: Python Modules and Packages](#)
 - [pip Documentation](#)
 - [Virtual Environments \(venv\)](#)
-

Module 7: File Handling

Goal: Learn how to read from and write to files, manage file paths, and work with structured data formats (CSV, JSON).

1. Why File Handling?

File handling allows programs to:

- Persist data beyond program execution.
 - Process large datasets stored in files.
 - Interact with configuration files, logs, and user data.
 - Exchange data with other applications using standard formats (CSV, JSON).
-

2. Working with Text Files

2.1. Opening a File

Use `open(filename, mode)` to return a file object.

Common modes:

Mode	Description
'r'	Read (default). File must exist.
'w'	Write. Creates new file or overwrites existing.
'a'	Append. Adds to end of file.
'r+'	Read and write.
'x'	Exclusive creation. Fails if file exists.
'b'	Binary mode (e.g., 'rb', 'wb').

Example:

```
file = open("example.txt", "r")
content = file.read()
file.close()
```

2.2. Reading Files

Method	Description
read()	Reads entire file as a string.
readline()	Reads one line.
readlines()	Returns list of lines.

```
with open("example.txt", "r") as file:
    entire = file.read()
    print(entire)
```

2.3. Writing Files

```
with open("output.txt", "w") as file:
    file.write("Hello, world!\n")
    file.write("Second line\n")
```

2.4. Appending to Files

```
with open("output.txt", "a") as file:
    file.write("Appended line\n")
```

2.5. The with Statement (Context Manager)

Using with ensures the file is properly closed even if an error occurs.

```
with open("data.txt", "r") as f:
    data = f.read()
# File is automatically closed
```

Alternative: Without with, you must manually close:

```
f = open("data.txt", "r")
data = f.read()
f.close()
```

Recommendation: Always use with.

3. Working with File Paths

3.1. Using os.path (Legacy)

```
import os

# Join paths
path = os.path.join("folder", "subfolder", "file.txt")

# Check existence
if os.path.exists(path):
    print("File exists")

# Get absolute path
abs_path = os.path.abspath("file.txt")

# Get directory and filename
dirname = os.path.dirname(path)
basename = os.path.basename(path)

# Split extension
name, ext = os.path.splitext("file.txt")  # ('file', '.txt')
```

3.2. Using pathlib (Modern, Recommended)

pathlib provides an object-oriented approach.

```
from pathlib import Path

# Create a Path object
path = Path("folder") / "subfolder" / "file.txt"

# Check existence
if path.exists():
    print("Exists")

# Read/write files easily
content = path.read_text()
path.write_text("New content")

# Get absolute path
abs_path = path.absolute()

# Get parts
print(path.parent)      # folder/subfolder
print(path.name)        # file.txt
print(path.stem)        # file
print(path.suffix)      # .txt
```

Advantages: Cross-platform, more readable, chainable methods.

4. Working with CSV Files

CSV (Comma-Separated Values) is a common data exchange format.

4.1. Reading CSV Files

```
import csv

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)    # row is a list of strings
```

If the file has a header row, you can use DictReader for dictionary access:

```
with open("data.csv", "r") as f:
    reader = csv.DictReader(f)    # first row becomes keys
    for row in reader:
        print(row["Name"], row["Age"])
```

4.2. Writing CSV Files

```
import csv

data = [
    ["Name", "Age", "City"],
    ["Alice", 30, "New York"],
    ["Bob", 25, "Los Angeles"]
]

with open("output.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

For dictionaries:

```
data = [
    {"Name": "Alice", "Age": 30, "City": "New York"},
    {"Name": "Bob", "Age": 25, "City": "Los Angeles"}
]

with open("output.csv", "w", newline="") as f:
    fieldnames = ["Name", "Age", "City"]
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

Note: Use `newline=""` to avoid extra blank lines on Windows.

5. Working with JSON Files

JSON (JavaScript Object Notation) is widely used for data interchange.

5.1. Reading JSON

```
import json

with open("data.json", "r") as f:
    data = json.load(f)    # parses JSON into Python dict/List
    print(data["name"])
```

5.2. Writing JSON

```
import json

data = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

with open("output.json", "w") as f:
    json.dump(data, f, indent=4)  # indent for pretty formatting
```

5.3. Working with JSON Strings

```
json_string = '{"name": "Alice", "age": 30}'
data = json.loads(json_string)  # string to Python object
output_string = json.dumps(data)  # Python object to JSON string
```

6. Handling File-Related Exceptions

Common exceptions:

- `FileNotFoundError`: file does not exist (when reading).
- `PermissionError`: insufficient permissions.
- `IsADirectoryError`: expected a file but got a directory.

Use try/except to handle gracefully:

```
try:
    with open("nonexistent.txt", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("File not found.")
except PermissionError:
    print("Permission denied.")
```

7. Hands-On Activities

Activity 1: Create and Read a Text File

Write a program that:

21. Asks the user for a filename and some lines of text (stop on empty input).
22. Writes those lines to the file.
23. Reads and displays the file content.

Solution:

```
filename = input("Enter filename: ")

# Write
with open(filename, "w") as f:
```

```
    print("Enter lines (empty line to stop):")
    while True:
        line = input()
        if not line:
            break
        f.write(line + "\n")

# Read
with open(filename, "r") as f:
    print("\nFile content:")
    print(f.read())
```

Activity 2: File Copy

Write a program that copies a source file to a destination file (use binary mode for any file type). Handle exceptions.

Solution:

```
src = input("Source file: ")
dst = input("Destination file: ")

try:
    with open(src, "rb") as source:
        with open(dst, "wb") as dest:
            dest.write(source.read())
    print("File copied successfully.")
except FileNotFoundError:
    print("Source file not found.")
except PermissionError:
    print("Permission denied.")
```

Activity 3: CSV Reader/Writer

Given a CSV file `students.csv` with columns: Name, Grade, read it, compute the average grade, and write a new file `summary.txt` with the average.

Solution:

```
import csv

total = 0
count = 0

with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        total += float(row["Grade"])
        count += 1

average = total / count if count > 0 else 0

with open("summary.txt", "w") as f:
    f.write(f"Number of students: {count}\\n")
    f.write(f"Average grade: {average:.2f}\\n")

print(f"Summary written to summary.txt")
```

Activity 4: JSON Configuration

Create a JSON file `config.json` with settings (e.g., `{"theme": "dark", "font_size": 14}`). Write a program that reads the config, allows the user to modify a setting, and saves it back.

Solution:

```
import json

filename = "config.json"
```

```

# Read existing config or create default
try:
    with open(filename, "r") as f:
        config = json.load(f)
except FileNotFoundError:
    config = {"theme": "light", "font_size": 12}

print("Current config:", config)

key = input("Enter setting to change (theme/font_size): ")
if key in config:
    value = input(f"New value for {key}: ")
    if key == "font_size":
        value = int(value)
    config[key] = value
    with open(filename, "w") as f:
        json.dump(config, f, indent=4)
    print("Config updated.")
else:
    print("Invalid setting.")

```

Activity 5: Directory Explorer with pathlib

Write a program that lists all .txt files in a given directory (and subdirectories recursively) and prints their sizes.

Solution:

```

from pathlib import Path

directory = input("Enter directory path: ")
path = Path(directory)

if not path.exists():
    print("Directory does not exist.")
else:
    for txt_file in path.rglob("*.txt"):
        size = txt_file.stat().st_size
        print(f"{txt_file} - {size} bytes")

```

8. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
FileNotFoundError	File path incorrect or file missing	Verify path; use <code>os.path.exists()</code> or <code>Path.exists()</code> before opening
UnicodeDecodeError	Trying to read binary file in text mode	Use binary mode ('rb') or specify encoding, e.g., <code>open(..., encoding='utf-8')</code>
Extra blank lines in CSV output	Using <code>newline=' '</code> missing on Windows	Always open CSV files with <code>newline=''</code>

Problem	Likely Cause	Solution
PermissionError	File is open in another program, or no write permission	Close other program; check file permissions
File not closed properly	Forgetting <code>close()</code> or not using <code>with</code>	Use <code>with</code> statement
JSONDecodeError	Invalid JSON format	Validate JSON using a linter; ensure proper quotes and syntax

9. Summary

- **File handling** enables persistent storage and data exchange.
- Use `open()` with modes (`r`, `w`, `a`, `b`, etc.) and always prefer the `with` context manager.
- **Paths** can be managed with `os.path` or the more modern `pathlib`.
- **CSV files** are handled by the `csv` module; use `DictReader`/`DictWriter` for dictionary access.
- **JSON files** are handled by the `json` module; use `load()/dump()` for files, `loads()/dumps()` for strings.
- Always handle file-related exceptions to make programs robust.

10. Additional Resources

- [Python Official: Input and Output](#)
- [Python `pathlib` Documentation](#)
- [Python `csv` Documentation](#)
- [Python `json` Documentation](#)
- [Real Python: Reading and Writing Files](#)

Module 8: Error and Exception Handling

Goal: Understand Python's exception handling mechanism and learn to write robust programs that gracefully manage errors.

1. Introduction to Errors and Exceptions

In Python, there are two main types of errors:

1.1. Syntax Errors

Errors that occur when the parser encounters invalid syntax. These prevent the program from running.

```
# SyntaxError: invalid syntax
print("Hello"
```

1.2. Exceptions (Runtime Errors)

Errors that occur during execution, even if the syntax is correct. They cause the program to stop unless handled.

```
# ZeroDivisionError
result = 10 / 0

# NameError
print(undefined_variable)

# TypeError
"2" + 2
```

2. Common Built-in Exceptions

Exception	Description
ZeroDivisionErr or	Division by zero.
NameError	Variable not defined.
TypeError	Operation applied to object of wrong type.
ValueError	Function receives argument of correct type but invalid value (e.g., <code>int("abc")</code>).
IndexError	Sequence index out of range.
KeyError	Dictionary key not found.
FileNotFoundErr or	File does not exist.
AttributeError	Object has no attribute.
ImportError	Module not found.

3. Handling Exceptions with try and except

3.1. Basic try/except

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

3.2. Catching Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input! Please enter a number.")
```

3.3. Catching Multiple Exceptions in One Block

```
except (ZeroDivisionError, ValueError) as e:
    print(f"Error: {e}")
```

3.4. Catching Any Exception (Not Recommended)

```
except Exception as e:  
    print(f"An error occurred: {e}")
```

Best Practice: Catch specific exceptions rather than a broad except.

3.5. Accessing Exception Details

```
try:  
    x = int("abc")  
except ValueError as e:  
    print(f"Error details: {e}")    # invalid literal for int() with base 10: 'abc'
```

4. else and finally Clauses

4.1. else Clause

Executes if no exception was raised in the try block.

```
try:  
    num = int(input("Enter a number: "))  
except ValueError:  
    print("Not a number!")  
else:  
    print(f"Great! You entered {num}.")
```

4.2. finally Clause

Executes **always**, regardless of whether an exception occurred or not. Used for cleanup (closing files, releasing resources).

```
try:  
    file = open("data.txt", "r")  
    content = file.read()  
except FileNotFoundError:  
    print("File not found.")  
finally:  
    print("Closing file.")  
    file.close()    # always executed
```

5. Raising Exceptions

Use raise to trigger an exception manually.

5.1. Raising Built-in Exceptions

```
def divide(a, b):  
    if b == 0:  
        raise ZeroDivisionError("Cannot divide by zero")  
    return a / b
```

5.2. Re-raising an Exception

```
try:  
    result = divide(10, 0)  
except ZeroDivisionError as e:
```



```
print("Logging error...")
raise # re-raise the same exception
```

6. Creating Custom Exceptions

Custom exceptions are user-defined classes that inherit from Exception.

```
class NegativeNumberError(Exception):
    """Raised when a negative number is encountered."""
    pass

def square_root(x):
    if x < 0:
        raise NegativeNumberError("Cannot take square root of negative number")
    return x ** 0.5

try:
    print(square_root(-4))
except NegativeNumberError as e:
    print(e)
```

Convention: Name custom exceptions with “Error” suffix and inherit from Exception (or a more specific subclass).

7. Best Practices for Exception Handling

- 24. **Be specific:** Catch only exceptions you expect.
 - 25. **Don't silence exceptions:** Avoid empty except: blocks.
 - 26. **Use finally for cleanup:** Close files, release resources.
 - 27. **Log exceptions:** In real applications, log errors for debugging.
 - 28. **Raise exceptions early:** Validate inputs and raise appropriate exceptions.
 - 29. **Use custom exceptions for application-specific errors.**
-

8. Hands-On Activities

Activity 1: Basic Input Validation

Write a program that asks the user for two numbers and divides them. Handle ZeroDivisionError and ValueError.

Solution:

```
while True:
    try:
        num1 = float(input("First number: "))
        num2 = float(input("Second number: "))
        result = num1 / num2
        print(f"Result: {result}")
        break
    except ValueError:
        print("Please enter valid numbers.")
```

```
except ZeroDivisionError:
    print("Cannot divide by zero. Try again.")
```

Activity 2: File Reader with Exception Handling

Create a program that reads a file given by the user and prints its content. Handle `FileNotFoundError` and `PermissionError`. Use `finally` to close the file.

Solution:

```
filename = input("Enter filename: ")
file = None
try:
    file = open(filename, "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("File not found.")
except PermissionError:
    print("Permission denied.")
finally:
    if file:
        file.close()
        print("File closed.")
```

Activity 3: Raising Exceptions in a Function

Write a function `withdraw(amount, balance)` that raises a `ValueError` if `amount` exceeds `balance` or if `amount` is negative. Test with a `try/except` block.

Solution:

```
def withdraw(amount, balance):
    if amount < 0:
        raise ValueError("Withdrawal amount cannot be negative")
    if amount > balance:
        raise ValueError("Insufficient funds")
    balance -= amount
    return balance

try:
    new_balance = withdraw(150, 100)
    print(f"New balance: {new_balance}")
except ValueError as e:
    print(f"Error: {e}")
```

Activity 4: Custom Exception

Create a custom exception `InvalidAgeError`. Write a program that asks for age and raises the exception if age is not between 0 and 120.

Solution:

```
class InvalidAgeError(Exception):
    pass

def validate_age(age):
```

```

    if age < 0 or age > 120:
        raise InvalidAgeError(f"Age {age} is out of valid range (0-120)")
    return True

try:
    age = int(input("Enter your age: "))
    validate_age(age)
    print("Valid age.")
except ValueError:
    print("Please enter a number.")
except InvalidAgeError as e:
    print(e)

```

Activity 5: Calculator with Comprehensive Exception Handling

Build a simple calculator that asks for two numbers and an operator. Use exception handling to catch division by zero, invalid operator, and input conversion errors.

Solution:

```

def calculator():
    try:
        num1 = float(input("First number: "))
        num2 = float(input("Second number: "))
        op = input("Operator (+, -, *, /): ")

        if op == '+':
            result = num1 + num2
        elif op == '-':
            result = num1 - num2
        elif op == '*':
            result = num1 * num2
        elif op == '/':
            if num2 == 0:
                raise ZeroDivisionError("Cannot divide by zero")
            result = num1 / num2
        else:
            raise ValueError(f"Unsupported operator: {op}")

        print(f"Result: {result}")
    except ValueError as e:
        print(f"Input error: {e}")
    except ZeroDivisionError as e:
        print(e)

calculator()

```

9. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
<code>except:</code> without exception type	Catching everything, hiding bugs	Always specify exception types

Problem	Likely Cause	Solution
Using <code>except</code> but missing subclasses	Works, but may catch too much	Still better than bare <code>except</code> , but be specific
<code>finally</code> block raising another exception	Overwrites original exception	Avoid raising in <code>finally</code> ; use <code>try</code> inside if needed
Not re-raising after logging	Silencing error after log	Use <code>raise</code> without arguments to preserve traceback
Custom exception not inheriting from <code>Exception</code>	Will not be caught by <code>except Exception</code>	Always inherit from <code>Exception</code> or a subclass

10. Summary

- **Exceptions** are runtime errors that can be handled to prevent crashes.
- Use `try/except` to catch exceptions.
- `else` runs if no exception, `finally` runs regardless.
- Raise exceptions with `raise` to signal errors.
- Create **custom exceptions** by subclassing `Exception`.
- Follow best practices: be specific, clean up resources, and don't silence errors.

11. Additional Resources

- [Python Official: Errors and Exceptions](#)
- [Real Python: Python Exceptions](#)
- [Python Exception Handling Best Practices](#)

Module 9: Object-Oriented Programming (OOP)

Goal: Understand the principles of object-oriented programming and apply them to create reusable, modular code.

1. Introduction to OOP

Object-Oriented Programming (OOP) is a paradigm that organizes code into **objects** that contain **data** (attributes) and **behavior** (methods). It helps:

- **Model real-world entities** (e.g., a Car has color and speed, can `drive()`).
- **Reuse code** through inheritance.
- **Encapsulate** complexity and hide implementation details.

Key concepts:

- **Class** – blueprint for creating objects.
 - **Object** – instance of a class.
 - **Attribute** – variable that belongs to an object/class.
 - **Method** – function defined inside a class.
-

2. Defining a Class and Creating Objects

2.1. Basic Class Definition

```
class Dog:
    """A simple Dog class."""
    pass

my_dog = Dog()           # create an instance
```

```
print(type(my_dog))      # <class '__main___.Dog'>
```

2.2. Adding Attributes

Attributes can be set directly on an object:

```
my_dog.name = "Buddy"
my_dog.age = 3
print(my_dog.name)       # Buddy
```

But this is not structured. Better to use `__init__`.

3. The `__init__` Method (Constructor)

`__init__` initializes an object's state when it is created.

```
class Dog:
    def __init__(self, name, age):
        self.name = name    # instance attribute
        self.age = age
```

```
# Creating objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)
```

```
print(dog1.name)         # Buddy
print(dog2.age)           # 5
```

- `self` refers to the instance being created.
 - Attributes set with `self` are **instance attributes** (each object has its own copy).
-

4. Instance Methods

Methods are functions defined inside a class that operate on the instance.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

    def get_human_years(self):
```

```
    return self.age * 7
```

```
dog = Dog("Buddy", 3)
dog.bark()           # Buddy says Woof!
print(dog.get_human_years())  # 21
```

5. Class Attributes and Class Methods

5.1. Class Attributes

Attributes shared by all instances of a class.

```
class Dog:
    species = "Canis familiaris"  # class attribute

    def __init__(self, name, age):
        self.name = name
        self.age = age

print(Dog.species)  # Canis familiaris (accessible via class)
dog = Dog("Buddy", 3)
print(dog.species)  # Also accessible via instance
```

5.2. Class Methods

Use @classmethod decorator. The first parameter is cls (the class itself).

```
class Dog:
    species = "Canis familiaris"

    @classmethod
    def get_species(cls):
        return cls.species

print(Dog.get_species())  # Canis familiaris
```

5.3. Static Methods

Use @staticmethod – no self or cls. Used for utility functions related to the class.

```
class Dog:
    @staticmethod
    def is_valid_age(age):
        return 0 < age < 30

print(Dog.is_valid_age(5))  # True
```

6. Inheritance

Inheritance allows a class (child) to reuse attributes and methods from another class (parent).

6.1. Basic Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclass must implement")

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

dog = Dog("Buddy")
cat = Cat("Whiskers")
print(dog.speak())    # Buddy says Woof!
print(cat.speak())    # Whiskers says Meow!
```

6.2. Using super()

super() calls a method from the parent class, useful to extend parent behavior.

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal {name} created.")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # call parent __init__
        self.breed = breed
        print(f"Dog of breed {breed} created.")

dog = Dog("Buddy", "Golden Retriever")
# Output:
# Animal Buddy created.
# Dog of breed Golden Retriever created.
```

6.3. Method Overriding

Child classes can override parent methods.

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"

# Dog.speak overrides Animal.speak
```

7. Encapsulation and Properties

Encapsulation hides internal data and provides controlled access through methods.

7.1. Name Mangling (Private Attributes)

Use `__` prefix to make an attribute “private” (name mangling).

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

```
acc = BankAccount(1000)
print(acc.get_balance())    # 1000
# print(acc.__balance)      # AttributeError
```

Note: Name mangling doesn’t enforce privacy, but it discourages direct access.

7.2. Properties (@property)

Use `@property` to define getters and setters for a more Pythonic approach.

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius    # protected attribute (convention)

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        return (self._celsius * 9/5) + 32
```

```
temp = Temperature(25)
print(temp.celsius)    # 25
temp.celsius = 30      # setter called
print(temp.fahrenheit) # 86.0
```

8. Magic (Dunder) Methods

Magic methods have double underscores (`__method__`). They define how objects behave with built-in operations.

Method	Description
<code>__init__(self, ...)</code>	Constructor
<code>__str__(self)</code>	String representation for end-users (<code>print()</code>)
<code>__repr__(self)</code>	Unambiguous representation for developers
<code>__len__(self)</code>	Returns length (used by <code>len()</code>)
<code>__add__(self, other)</code>	Defines behavior for <code>+</code>
<code>__eq__(self, other)</code>	Defines behavior for <code>==</code>

Example:

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def __str__(self):
        return f"{self.title} by {self.author}"

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}', {self.pages})"

    def __len__(self):
        return self.pages

book = Book("1984", "George Orwell", 328)
print(book)           # 1984 by George Orwell (__str__)
print(repr(book))     # Book('1984', 'George Orwell', 328)
print(len(book))      # 328
```

9. Hands-On Activities

Activity 1: Simple Class – Rectangle

Create a class `Rectangle` with attributes `width` and `height`. Include methods:

- `area()` returns area.
- `perimeter()` returns perimeter.
- `__str__` returns a readable description.

Solution:

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def __str__(self):
        return f"Rectangle(width={self.width}, height={self.height})"

rect = Rectangle(5, 3)
```

```
print(rect)                # Rectangle(wi
dth=5, height=3)
```

```
print(rect.area())        # 15
print(rect.perimeter())   # 16
```

Activity 2: Inheritance – Vehicle Hierarchy

Create a base class Vehicle with make, model, and a method info(). Create subclasses Car and Motorcycle. Car has num_doors, Motorcycle has has_sidecar. Override info() to include specific details.

Solution:

```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def info(self):
        return f"{self.make} {self.model}"

class Car(Vehicle):
    def __init__(self, make, model, num_d
doors):
        super().__init__(make, model)
        self.num_doors = num_doors
```

```
    def info(self):
        return f"{super().info()}, {self.
num_doors} doors"

class Motorcycle(Vehicle):
    def __init__(self, make, model, has_s
idecar):
        super().__init__(make, model)
        self.has_sidecar = has_sidecar

    def info(self):
        sidecar = "with sidecar" if self.
has_sidecar else "no sidecar"
        return f"{super().info()}, {sidec
ar}"

car = Car("Toyota", "Camry", 4)
bike = Motorcycle("Harley", "Sportster",
False)
print(car.info())        # Toyota Camry, 4 d
doors
print(bike.info())       # Harley Sportster,
no sidecar
```

Activity 3: Encapsulation – Bank Account

Create a class BankAccount with a private attribute __balance. Provide methods:

- deposit(amount) adds if positive.
- withdraw(amount) subtracts if sufficient funds.
- get_balance() returns balance.
- Use property to allow read-only access to balance.

Solution:

```
class BankAccount:
    def __init__(self, initial_balance=0)
:
        self.__balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
```

```
        self.__balance += amount
        return True
    return False

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return True
        return False

    @property
    def balance(self):
        return self.__balance
```

```
acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(200)
print(acc.balance)    # 1300
```

Activity 4: Magic Methods – Vector Class

Create a class Vector that represents a 2D vector. Implement:

- __init__(x, y)
- __add__(other) returns new Vector.

- `__sub__(other)`
- `__mul__(scalar)`
- `__eq__(other)`
- `__str__` and `__repr__`

Solution:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
```

```
        return Vector(self.x * scalar, self.y * scalar)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
v3 = v1 + v2
print(v3)                # Vector(6, 8)
print(v1 * 3)             # Vector(6, 9)
print(v1 == Vector(2, 3)) # True
```

Activity 5: Class Method and Static Method

Create a class Person with:

- Instance attributes: name, age.
- Class attribute: population (count of instances).
- `__init__` increments population.
- Class method `get_population()`.
- Static method `is_adult(age)` returns True if age >= 18.

Solution:

```
class Person:
    population = 0

    def __init__(self, name, age):
        self.name = name
```

```
        self.age = age
        Person.population += 1

    @classmethod
    def get_population(cls):
        return cls.population

    @staticmethod
    def is_adult(age):
        return age >= 18

p1 = Person("Alice", 25)
p2 = Person("Bob", 17)
print(Person.get_population()) # 2
print(Person.is_adult(20))    # True
print(Person.is_adult(16))    # False
```

10. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
<code>TypeError: __init__() missing 1 required positional argument</code>	Forgetting to pass required arguments when creating object	Check class <code>__init__</code> parameters; provide all required args
<code>AttributeError: object has no attribute __balance</code>	Trying to access private attribute directly	Use getter/setter or property, not direct name

Problem	Likely Cause	Solution
Infinite recursion in property setter	Setter calls itself (e.g., <code>self.balance = value</code> inside setter)	Use a different internal name (e.g., <code>_balance</code>)
Method overriding but forgetting to call <code>super()</code>	Losing parent initialization	Call <code>super().__init__()</code> if parent needs setup
<code>self</code> missing in method definition	Forgetting <code>self</code> as first parameter	Always include <code>self</code> for instance methods

11. Summary

- **Classes** are blueprints; **objects** are instances.
- `__init__` initializes instance attributes.
- **Instance methods** operate on an object; **class methods** operate on the class.
- **Inheritance** allows reuse and extension.
- **Encapsulation** hides internal data; use properties (`@property`) for controlled access.
- **Magic methods** enable custom behavior for built-in operations.
- OOP helps structure code, reduce duplication, and model real-world concepts.

12. Additional Resources

- [Python Official: Classes](#)
- [Real Python: Object-Oriented Programming](#)
- [Python property Documentation](#)
- [Magic Methods Guide](#)

Module 10: Intermediate Python Features

Goal: Master advanced Python features that make code more concise, expressive, and efficient.

1. Comprehensions

Comprehensions provide a compact way to create collections from existing iterables.

1.1. List Comprehensions

```
# Basic: [expression for item in iterable if condition]
squares = [x**2 for x in range(10)] # [0,1,4,9,16,25,36,49,64,81]
evens = [x for x in range(20) if x % 2 == 0] # [0,2,4,...,18]
```

1.2. Dictionary Comprehensions

```
# {key_expression: value_expression for item in iterable}
squares_dict = {x: x**2 for x in range(5)} # {0:0, 1:1, 2:4, 3:9, 4:16}
```

1.3. Set Comprehensions

```
# {expression for item in iterable}
unique_lengths = {len(word) for word in ["apple", "banana", "cherry"]} # {5,6}
```

1.4. Nested Comprehensions

```
# Flatten a 2D List
matrix = [[1, 2], [3, 4], [5, 6]]
flattened = [num for row in matrix for num in row] # [1,2,3,4,5,6]
```

2. Generator Expressions

Generator expressions are similar to list comprehensions but return an iterator that yields values lazily (one at a time), saving memory.

```
# Generator expression (parentheses, not brackets)
gen = (x**2 for x in range(10)) # returns generator object

# Iterate (values computed on demand)
for value in gen:
    print(value) # prints 0,1,4,... but not stored in memory all at once
```

Use case: Large datasets where you don't need the entire list in memory.

Conversion: `list(gen)` if you need all values at once.

3. args and *kwargs

These allow functions to accept a variable number of arguments.

3.1. args (Variable Positional Arguments)

Collects extra positional arguments into a tuple.

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4)) # 10
```

3.2. *kwargs (Variable Keyword Arguments)

Collects extra keyword arguments into a dictionary.

```
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="New York")
```

3.3. Unpacking in Function Calls

Use `*` and `**` to unpack iterables/dictionaries into function arguments.

```
def add(a, b, c):  
    return a + b + c  
  
numbers = [1, 2, 3]  
print(add(*numbers))    # 6  
  
kwargs = {'a': 1, 'b': 2, 'c': 3}  
print(add(**kwargs))    # 6
```

4. Map, Filter, Reduce

These functional programming tools operate on iterables.

4.1. `map(function, iterable, ...)`

Applies function to every item and returns an iterator.

```
nums = [1, 2, 3, 4]  
squared = map(lambda x: x**2, nums)  
print(list(squared))    # [1, 4, 9, 16]
```

4.2. `filter(function, iterable)`

Returns an iterator with items where function returns True.

```
evens = filter(lambda x: x % 2 == 0, nums)  
print(list(evens))    # [2, 4]
```

4.3. `reduce(function, iterable[, initializer])`

Reduces the iterable to a single value by applying function cumulatively (from `functools`).

```
from functools import reduce
```

```
product = reduce(lambda x, y: x * y, nums)    # 1*2*3*4 = 24  
print(product)
```

```
# With initializer
```

```
product = reduce(lambda x, y: x * y, nums, 10)    # 10 * 1 * 2 * 3 * 4 = 240
```

Comparison with comprehensions: Comprehensions are often more readable. Use `map`/`filter` when you already have a named function or prefer functional style.

5. Decorators

Decorators are functions that modify the behavior of other functions without changing their source code.

5.1. Simple Decorator

```
def timer(func):  
    import time
```

```

def wrapper(*args, **kwargs):
    start = time.time()
    result = func(*args, **kwargs)
    end = time.time()
    print(f"{func.__name__} took {end - start:.4f} seconds")
    return result
return wrapper

@timer
def slow_function(delay):
    time.sleep(delay)
    return "Done"

print(slow_function(2))

```

5.2. Decorator with Arguments

```

def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            results = []
            for _ in range(n):
                results.append(func(*args, **kwargs))
            return results
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    return f"Hello, {name}"

print(greet("Alice"))  # ['Hello, Alice', 'Hello, Alice', 'Hello, Alice']

```

5.3. Preserving Metadata with `functools.wraps`

Always use `@wraps` to copy metadata from the original function to the wrapper.

```

from functools import wraps

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        """Wrapper docstring"""
        return func(*args, **kwargs)
    return wrapper

```

6. Custom Context Managers

Context managers allow you to allocate and release resources precisely (like files, locks). Use the `with` statement.

6.1. Using Class with `__enter__` and `__exit__`

```

class ManagedFile:
    def __init__(self, filename, mode):
        self.filename = filename

```

```

        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        # Return True to suppress exception, False to propagate
        return False

with ManagedFile("hello.txt", "w") as f:
    f.write("Hello, world!")

```

6.2. Using contextlib.contextmanager (Generator-Based)

```
from contextlib import contextmanager
```

```

@contextmanager
def managed_file(filename, mode):
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()

with managed_file("hello.txt", "w") as f:
    f.write("Hello, world!")

```

7. Hands-On Activities

Activity 1: Comprehensions

30. Create a list of squares of even numbers from 0 to 20.
31. Create a dictionary mapping numbers to their cubes for numbers 1–10.
32. Create a set of all unique characters in the string "abracadabra".

Solution:

```

# 1
even_squares = [x**2 for x in range(0, 21, 2)]
# 2
cubes_dict = {x: x**3 for x in range(1, 11)}
# 3
unique_chars = {ch for ch in "abracadabra"}

```

Activity 2: Generator Expression

Write a generator expression that yields the first 10 Fibonacci numbers. Use it to print them without storing all at once.

Solution:


```
def fib_generator(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

# Generator expression alternative (though not typical for Fibonacci)
# This is a simple demonstration:
fib_gen = (fib for fib in [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]) # hardcoded
# Better: define generator function and iterate
for val in fib_generator(10):
    print(val)
```

Activity 3: Flexible Function with args and *kwargs

Write a function create_person that accepts:

- args – any number of strings (names)
- *kwargs – any number of attributes (e.g., age, city) It should print each name and then the attributes.

Solution:

```
def create_person(*args, **kwargs):
    print("Names:", args)
    print("Attributes:")
    for key, value in kwargs.items():
        print(f" {key}: {value}")

create_person("Alice", "Bob", age=25, city="NYC")
```

Activity 4: Map, Filter, Reduce

Given a list of numbers, use:

- map to square them.
- filter to keep only those > 10.
- reduce to find the product of the remaining numbers.

Solution:

```
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]
squared = list(map(lambda x: x**2, numbers))
filtered = list(filter(lambda x: x > 10, squared))
product = reduce(lambda x, y: x * y, filtered)
print(product)
```

Activity 5: Decorator for Logging

Write a decorator log that prints the function name and arguments before calling the function. Apply it to a few functions.

Solution:

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper
```

```
@log
def add(a, b):
    return a + b
```

```
@log
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
```

```
add(3, 5)
greet("Alice", greeting="Hi")
```

Activity 6: Custom Context Manager

Create a context manager Timer that measures how long a block of code takes. Use it with with.

Solution (class-based):

```
import time

class Timer:
    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.time()
        print(f"Elapsed: {self.end - self.start:.4f} seconds")

with Timer():
    time.sleep(2)
```

Solution (generator-based):

```
from contextlib import contextmanager
import time

@contextmanager
def timer():
    start = time.time()
    try:
        yield
    finally:
        end = time.time()
        print(f"Elapsed: {end - start:.4f} seconds")

with timer():
    time.sleep(2)
```

8. Common Pitfalls & Troubleshooting

Problem	Likely Cause	Solution
List comprehension uses too much memory	Storing large intermediate list	Use generator expression instead
*args / **kwargs order in function definition	Must be placed after positional parameters	Order: <code>def func(a, b, *args, **kwargs)</code>
Forgetting to unpack with * in call	Passing a list as a single argument	Use <code>*list</code> to unpack
map returns iterator, not list	Python 3 behavior	Wrap with <code>list()</code> if needed
Decorator loses original function metadata	Not using <code>@wraps</code>	Always import and use <code>functools.wraps</code>
Context manager doesn't close resource on exception	Forgetting to handle in <code>__exit__</code>	Ensure <code>__exit__</code> closes/cleans up; use <code>try/finally</code> in generator version
reduce not imported	It's in <code>functools</code>	<code>from functools import reduce</code>

9. Summary

- **Comprehensions** provide concise ways to build lists, dicts, and sets.
 - **Generator expressions** are memory-efficient for large data.
 - **args and kwargs** allow functions to accept variable arguments.
 - **map, filter, reduce** enable functional programming styles.
 - **Decorators** modify function behavior elegantly.
 - **Custom context managers** give fine-grained resource control.
-

10. Additional Resources

- [Python Official: List Comprehensions](#)
 - [Real Python: Python Generators](#)
 - [Python Official: args and kwargs](#)
 - [Real Python: Decorators](#)
 - [Python contextlib Documentation](#)
-